# Project Documentation: Hospital Appointment Booking System

**Project Title**

**Hospital Appointment Booking System**

**Course**

**Programming Fundamentals**

**Purpose of the Project**

In today's fast-paced world, hospitals need an efficient system to manage appointments without manual errors or delays. This project simulates a real-world Hospital Appointment Booking System using a console-based C++ program. It provides a comprehensive system where patients can book, view, and cancel appointments, doctors can manage their schedules, and administrators can control and oversee the entire system.

This project was developed with the purpose of:

- Demonstrating real-world problem-solving using programming.
- Creating a simple, user-friendly system for different users (admin, doctors, and patients).
- Reducing reliance on manual scheduling methods.
- Teaching how to organize and structure data in a program using C++.

It mimics the operational workflow of hospitals and clinics and can be expanded into a full software product in the future.

**Objectives**

The main objectives of this project are:

- To apply **programming fundamentals** in a meaningful project.
- To demonstrate the use of **structures, arrays, vectors**, and **modular functions**.
- To provide an understanding of **user authentication** and role-based access.
- To simulate a **real-life system** that can be further enhanced with object-oriented design and database support.
- To help the developer understand how to handle multiple entities (patients, doctors, appointments) in a single application.

This project aims to bridge the gap between theory and practice by offering an experience of

building a practical system.

**Key Features**

**1. Role-Based Login System**

- The application has **three roles**:
  - **Administrator**
  - **Doctor**
  - **Patient**
- Each role has a separate interface and functionality.
- Admin credentials:
  - Username: Cybrixx5, saher, or isbah
  - Passwords: 19.95, zaheer, or rizwan

**2. Admin Panel**

Admins have the most control in the system. They can:

- Add new doctors with details like name, ID, specialization, availability, and schedule.
- View the entire list of doctors.
- View all appointments scheduled by patients.
- Register new patients by setting up their usernames, passwords, and full names.

**3. Doctor Panel**

Doctors can:

- Log in securely using their credentials.
- View appointments booked with them.
- Cancel appointments of specific patients by entering the patient's ID.

**4. Patient Panel**

Patients can:

- View available doctors, including their specialties and working days.
- Book appointments with a doctor based on date and availability.
- Cancel appointments they no longer wish to attend.
- View their upcoming appointments.

**5. Data Structures Used**

- struct Doctor is used to store information such as ID, name, specialization, available time, and days.
- struct Appointment stores data related to booked appointments like patient ID, doctor ID, date, and day.

- Arrays and vectors are used to manage multiple records.
- Strings handle usernames, passwords, and names.

### Modules and Functionalities

#### 1. Login Module

- Validates login for all three user types.
- Ensures access is granted only after correct credentials are entered.
- Displays a customized welcome message.

#### 2. Appointment Booking Module

- Patients can book an appointment by entering the doctor ID, date, and day.
- Before booking, the system checks if the doctor is available on the selected day.
- The appointment is stored and becomes visible in the system.

#### 3. Appointment Viewing Module

- Patients can view only their own appointments.
- Doctors can view all appointments booked with them.
- Admins can view every appointment in the system.

#### 4. Appointment Cancellation Module

- Patients can cancel their own appointments.
- Doctors can cancel an appointment by providing the patient ID.
- Once canceled, the appointment is removed from the system.

#### 5. Doctor Management Module

- Admins can add doctor details, including:
  - Unique ID
  - Full name
  - Specialization
  - Available time
  - Available days
- This information helps patients choose doctors more efficiently.

#### 6. Patient Registration Module

- Admins can register new patients.
- Each patient is assigned:
  - A unique username
  - A password
  - Their full name
- The system supports registration for up to 100 patients.

### Technologies Used

- **Programming Language**: C++
- **Concepts Used**:
  - Structures
  - Arrays
  - Vectors
  - Loops (for and while)
  - Conditional statements (if-else, switch-case)
  - Functions and modular design
  - Console input/output operations

### System Limitations

While the system is well-organized and effective for basic functionality, it does have a few limitations:

- It does **not save data permanently** — there is no file handling or database integration.
- All login information is **hardcoded**, except for patients added by admins.
- There is **no conflict checking** for overlapping appointments or timing issues.
- The user interface is **text-only** with no graphics or GUI.
- The system can handle **up to 100 entries** for doctors and patients due to array limitations.

### Advantages

This system has several educational and practical benefits:

- Helps learners understand **real-world problem solving**.
- Provides experience with **data structures and modular programming**.
- Introduces the idea of **multi-user systems** and access management.
- Prepares the developer for more advanced topics like OOP, file handling, and databases.
- Encourages writing **readable, maintainable, and structured code**.

### Future Improvements

The system can be greatly improved and expanded in the following ways:

1. **Implement File Handling**
   - Save and load data from external files so that information persists after the program ends.

2. **Use Object-Oriented Programming**
    - Convert structures into classes.
    - Use concepts like encapsulation and inheritance.
3. **Add Appointment Conflict Detection**
    - Check if a doctor already has an appointment at a particular time before allowing a new booking.
4. **GUI Interface**
    - Create a graphical user interface using libraries like Qt, SFML, or even web technologies.
5. **Database Integration**
    - Use SQL or any database system to manage large volumes of data efficiently.
6. **Search and Filter**
    - Allow users to search for doctors by name, ID, or specialization.
    - Add filtering options for appointments.
7. **Notification System**
    - Send reminders via email or SMS to patients and doctors for upcoming appointments.

**Conclusion**

The Hospital Appointment Booking System is a complete educational project that introduces core concepts of C++ programming. By combining data structures, user interfaces, and role-based access, it reflects how real systems operate behind the scenes.

This system not only fulfills the requirements of a programming fundamentals course but also serves as a stepping stone toward more professional software development. It offers immense scope for future enhancements and demonstrates a solid understanding of structured programming and system design